ShinySwarm Benchmark Report

Kafka Architecture (5-run aggregate)

Iva Gunjaca — University of Amsterdam

June 2026

1 Test Environment

Parameter	Value
Server	Hetzner CX32 — 8 vCPU, 16 GB RAM
Kubernetes	K3s single-node
Kafka	KRaft mode 3.7.0 (no ZooKeeper)
Backend	Spring Boot, PostgreSQL, Redis
Frontend	Angular + nginx reverse proxy
Load generator	k6 (remote)
Apps under test	8 collaborative R Shiny applications
Runs aggregated	5 (run-1 . . . run-5)

Validity Notes (read first)

- **Throughput (test 04) is excluded.** Across the 5 runs its p95 rose monotonically (5.8 s → 52.4 s) and error rate rose 5% → 30%, because the server was not reset between runs. The runs are serially contaminated, not independent, so the test is not reported here.
- **Latency tests lack a per-VU breakdown.** Tests 01, 02, 03 and 08 ramp virtual users but tag metrics only with `app_name`, not `vu_bucket`. Their avg/median/p95 therefore aggregate all load levels together: *the median reflects low-VU samples while the p95 reflects high-VU samples.* Read median as the typical case and p95 as the loaded tail, not as a single operating point.
- **Per-app thresholds are baseline-calibrated** (10× each app's single-user baseline). A PASS means the app stayed within its own expected envelope under load, *not* that it was fast in absolute terms.

2 Summary

#	Benchmark	Result	Key Finding (5-run mean)
01	State Relay Latency	WARN	6/8 apps ≈ 265 ms; Analytics & MLTrainer ≈ 10.2 s (intrinsic compute)
02	Collaboration Latency	WARN	6/8 apps ≈ 260 ms; Analytics & MLTrainer 13–14 s
03	Save & Restore Latency	PASS	Save ≈ 9 ms, all apps within threshold
04	Throughput	**EXCLUDED — invalid (see validity notes)**	
05	Data Loss	PASS	0% loss, all runs, all apps
06	Cross-Contamination	PASS	0 contamination events, all runs
07	Session Lifecycle	PASS	0 lifecycle errors; op median ≈ 0.7 s
08	Multi-User Collaboration	WARN	6/8 apps ≈ 280 ms; Analytics & MLTrainer 16–17 s
09	WebSocket Presence	PASS	connect ≈ 4 ms median, stable across VUs

Headline. On the collaboration path, six of eight apps relay state in roughly a quarter of a second. The two outliers, Analytics and MLTrainer, sit near 10 s. This is *intrinsic R computation time for those apps*, not a Kafka transport cost: the POST into Kafka is ≈ 7 ms; the remaining time is the consumer waiting for the app's own computation to finish (see §**??**).

3 Detailed Results

3.1 01 — State Relay Latency

Round-trip: POST state → Kafka → R consumer computes → GET result. Per-app `relay_total_ms`, mean of 5 runs.

App	avg	median	p95	threshold
Calculator	261 ms	264 ms	282 ms	PASS
Geospatial	265 ms	264 ms	271 ms	PASS
DataExchange	264 ms	264 ms	273 ms	PASS
MonteCarlo	264 ms	264 ms	272 ms	PASS
Advanced	265 ms	264 ms	278 ms	PASS
ClimateAnomaly	267 ms	265 ms	278 ms	PASS
Analytics	10.2 s	10.2 s	10.3 s	FAIL
MLTrainer	10.0 s	10.2 s	10.3 s	PASS*

*MLTrainer passes only because its threshold is baseline-calibrated to its ≈ 14 s single-user compute (SLO 143 s). Analytics fails its 830 ms SLO.

Consistency: across the 5 runs the per-app p95 varies by $< 2\%$ — the split between the fast six and the slow two is structural, not noise.

3.2 Transport vs. Computation

The relay metric decomposes into the POST (push into Kafka) and the POLL (wait for the computed result). Global medians/p95 (run-1, representative):

Stage	median	p95
`relay_post_ms` (POST into Kafka)	5 ms	11 ms
`http_req_duration` (HTTP layer)	4 ms	45 ms
`relay_poll_ms` (wait for result)	261 ms	10,450 ms

The POST is ≈ 7 ms regardless of app. The entire latency difference between apps lives in the POLL, i.e. in how long each app's R code takes to run. Kafka is not the bottleneck for the heavy apps; their own computation is.

3.3 02 — Collaboration Latency

Host posts state; guest polls until the update appears. Per-app `collab_propagation_ms`, mean of 5 runs.

App	avg	median	p95	threshold
Calculator	257 ms	265 ms	494 ms	**PASS**
Geospatial	257 ms	266 ms	478 ms	**PASS**
DataExchange	255 ms	267 ms	457 ms	**PASS**
MonteCarlo	284 ms	281 ms	552 ms	**PASS**
Advanced	243 ms	259 ms	472 ms	**PASS**
ClimateAnomaly	292 ms	281 ms	593 ms	**PASS**
Analytics	14.0 s	13.5 s	20.5 s	**FAIL**
MLTrainer	14.2 s	13.0 s	22.0 s	**PASS***

Same pattern as relay: six light apps propagate in <300 ms median; Analytics and MLTrainer are dominated by their compute.

3.4 03 — Save & Restore Latency

PostgreSQL snapshot cycle. Per-app `save_to_db_ms`, mean of 5 runs.

App	avg	median	p95	threshold
Calculator	9 ms	9 ms	12 ms	**PASS**
Geospatial	9 ms	9 ms	14 ms	**PASS**
DataExchange	10 ms	9 ms	13 ms	**PASS**
MonteCarlo	9 ms	9 ms	12 ms	**PASS**
Advanced	9 ms	9 ms	12 ms	**PASS**
ClimateAnomaly	9 ms	9 ms	12 ms	**PASS**
Analytics	7 ms	7 ms	10 ms	**PASS**
MLTrainer	7 ms	7 ms	10 ms	**PASS**

Persistence bypasses the Kafka consumer path, so it is uniform and fast across all apps, including the two heavy ones. Confirms the bottleneck is isolated to computation, not storage.

3.5 05 — Data Loss

Metric	Value (all 5 runs)
Loss rate	0%
Checks passed	120 / 120 (100%) per run

Threshold `dataloss_pct` $<$ `5%`: **PASS**.

3.6 06 — Cross-Contamination

Metric	Value (all 5 runs)
Contamination events	0

Threshold `contamination_events == 0`: **PASS**. State isolation holds across all parallel sessions.

3.7 07 — Session Lifecycle

Metric	Value (run-1, representative)
Lifecycle errors	0 / 4,086 (0%)
`lifecycle_op_ms` median	698 ms
`lifecycle_op_ms` p95	3.61 s

Advanced crosses its per-app SLO in some runs; all other apps pass. Overall checks 100%.

3.8 08 — Multi-User Collaboration

One host posts; multiple guests poll. Per-app `multi_propagation_all_ms`, mean of 5 runs.

App	avg	median	p95	threshold
Calculator	285 ms	328 ms	328 ms	**PASS**
Geospatial	273 ms	281 ms	281 ms	**PASS**
DataExchange	269 ms	283 ms	283 ms	**PASS**
MonteCarlo	266 ms	290 ms	290 ms	**PASS**
Advanced	266 ms	291 ms	291 ms	**PASS**
ClimateAnomaly	295 ms	293 ms	293 ms	**PASS**
Analytics	17.5 s	17.2 s	17.2 s	**FAIL**
MLTrainer	16.2 s	16.1 s	16.1 s	**PASS***

3.9 09 — WebSocket Presence

STOMP-over-WebSocket connect and JOIN round-trip. Has a per-VU breakdown.

Metric	median	p95	vs. VU load
`ws_connect_ms`	4 ms	6 ms	flat (no degradation)
`ws_join_roundtrip_ms`	5 ms	7 ms	flat

Connect time stays ≈ 4 ms from 1 to 50 VUs.

4 Analysis

The Kafka architecture is correct and stable on every reliability dimension: zero data loss, zero cross-contamination, zero lifecycle errors, across all five runs. Persistence (save/restore) and presence (WebSocket) are uniformly fast.

The collaboration-path latency splits the apps cleanly into two groups. Six apps (Calculator, Geospatial, DataExchange, MonteCarlo, Advanced, ClimateAnomaly) relay in ≈ 265 ms. Two apps (Analytics, MLTrainer) sit near 10 s on relay and higher on multi-user. The transport/computation decomposition (§**??**) shows the POST into Kafka is ≈ 7 ms for every app; the difference is entirely in the POLL, i.e. the time each app's R backend needs to compute its result. **For Analytics and MLTrainer this latency is intrinsic to the application, not a property of Kafka.**

Open question for the next experiment. Because all apps currently share consumer infrastructure, a heavy app's long computation may head-of-line block lighter apps under concurrency. Whether per-app topic partitioning with key-based routing isolates that effect is the subject of the planned follow-up run. It is an *isolation-under-concurrency* question; it will not (and is not expected to) reduce the heavy apps' own compute time.

Caveats. (1) Throughput is excluded as invalid. (2) Latency tests aggregate across VU levels because they are not VU-tagged; a future run should tag tests 01/02/03/08 with `vu_bucket` to recover latency-vs-load curves for the collaboration path.